

biA Privacy Preserving System for Collecting and Reporting Sensitive Diversity Survey Data Using Paillier Encryption

Author: Ben Jefferson, BAFTA Media Technology

Version: 1.2

Date: 3/9/2025

Contents

1. Abstract.....	2
2. Introduction.....	2
3. System Architecture.....	5
4. Data Processing Workflow.....	6
5. Reporting Mechanisms.....	7
6. Additional Safeguard: Requester Key Pairs And Encrypted Results.....	11
7. Protecting Against Inference In Aggregate Reporting.....	12
8. Defence Against Differencing Attacks.....	14
9. Security Analysis.....	16
10. Fully Mitigated Attacks.....	17
11. Performance And Scalability.....	18
12. Comparison with Alternative Approaches.....	20
13. Next Steps.....	21
14. Conclusion.....	22
15. Actors & keys.....	23
16. Static setup.....	23
17. Message flow (per reporting period / cohort).....	23

1. Abstract

This paper presents a privacy preserving system for collecting and reporting on sensitive diversity survey data which provides security against attack under a Semi-Honest Adversary Model. The proposed architecture leverages Paillier encryption, combined with conventional asymmetric encryption, and existing differential privacy techniques to protect individual responses while allowing flexible and detailed reporting on the underlying data.

The system is designed to balance security with practicality, acknowledging certain risks due to the small dataset size and business requirements for flexible reporting. We discuss the system's components, data processing workflows, security considerations, and mitigation strategies for potential vulnerabilities.

2. Introduction

2.1. Background

The collection and analysis of diversity data are essential for organisations aiming to foster inclusive environments. However, handling sensitive personal information presents significant privacy challenges. Protecting individual responses while enabling meaningful analysis requires innovative approaches that balance security and practicality.

2.2. Problem Statement

Traditional methods of collecting and reporting survey data often expose individual responses to risks of unauthorised access or inference attacks. Many current implementations rely on symmetric encryption of response data which offers little protection against anything other than physical theft of the underlying storage medium - something that almost never happens. More advanced systems do exist which use asymmetric encryption, but these are still vulnerable to compromise of the server holding the private key. The most advanced systems use homomorphic asymmetric encryption but in most implementations this is extremely computationally intensive.

2.3. Objectives

This paper proposes a secure system that:

- Protects individual survey responses using Paillier encryption.
- Allows flexible and detailed reporting on aggregated data.
- Balances security with practical considerations for small datasets.
- Mitigates risks associated with single-server compromises.

2.4. Scope Of This Paper

This paper focuses exclusively on the encryption architecture designed for the secure collection and reporting of sensitive diversity survey data. It details the cryptographic methods, system components, and data processing workflows essential for protecting individual responses within the proposed framework. It is important to note that the final

implementation of this system will incorporate a multitude of additional technical and procedural security safeguards not covered in this document. These safeguards include, but are not limited to, comprehensive access controls, authentication mechanisms, data integrity verification, compliance with relevant legal and regulatory requirements, and robust organisational policies. These measures will work in tandem with the encryption architecture to ensure end-to-end security, privacy and legal compliance throughout the data lifecycle.

This paper also does not seek to address aspects such as how each user is identified and what process is used to seek consent. The omission of these from this paper does not mean that these are not important factors, simply that they are not the focus of this document.

This paper covers the storage of radio-button and checkbox style questions which make up the majority of the survey. It does not cover textual answers such as might crop up where there is an “Other” option offered in the multiple choice answer set. It is also likely that any age/date of birth/year of birth questions will need specific handling. Once again, these need careful consideration, but are not included here.

The approach described in this paper relies on collecting diversity data directly from users in their web browser, with JavaScript code performing Paillier encryption before any data leaves the browser. It should be noted that this client-side stage is, in practice, the most exposed part of the system. An attacker could, for example, redirect traffic to a malicious look-alike site that bypasses encryption, or compromise the legitimate server and substitute JavaScript that silently exfiltrates unprotected data. There are well-established technologies and operational processes to mitigate these threats - such as DNSSEC, TLS, code signing, content integrity checks, and secure software supply chain practices - but their design and deployment are outside the scope of this paper.

2.5. The Importance of Comprehensibility

We believe that a practical requirement in this domain is that survey participants must be able to understand, at least in outline, how their data is being protected. Participation in diversity surveys is voluntary, and willingness to contribute is closely linked to trust.

In our experience, systems that cannot be explained in plain language to inquisitive but non-technical users risk losing credibility and adoption. The protection of responses must therefore not only be strong in a formal cryptographic sense, but also explainable at a human level.

This principle has influenced several of the design choices in the present proposal. We have sought to use cryptographic constructions that deliver meaningful additional protection compared with current practice, while avoiding mechanisms that would make the system opaque to stakeholders. Paillier encryption, in particular, represents a balance: it provides genuine homomorphic security benefits while remaining just about comprehensible to non-specialists. More elaborate techniques (for example, threshold cryptography or zero-knowledge proofs) may eventually be introduced, but these carry the risk of reducing transparency and explainability.

In short, in our view, comprehensibility is not an afterthought but a fundamental design constraint, on a par with efficiency and security.

3. System Architecture

3.1. Components

The proposed system comprises three servers:

3.1.1. Data Server

- Receives and stores encrypted survey responses.
- Manages user identifiers
- Performs initial data aggregation using Paillier addition of encrypted answers before passing the tallies to the reporting server for decryption.

3.1.2. Reporting Server

- Decrypts aggregated data sums.
- Returns decrypted sums to the data server for final tallying.

3.1.3. Intersection Server

- Determines whether individual responses meet specific criteria without learning the actual data.
- This is achieved by decrypting offset-adjusted answers to specific questions in order to evaluate logic expressions which determine inclusion or exclusion.

3.2. Overview

The reporting and intersection servers each have their own Paillier key pair, with private keys stored securely on the respective servers. The servers are operated by separate agencies to prevent (or at least complicate) collusion. Thus the data is protected from compromise of any single server:

- If the **data server** is compromised all the data is encrypted and cannot be decrypted without the private key either from the reporting or the intersection server
- If the **reporting server** is compromised this only ever receives aggregate tallies from the data server and even then it has no idea what characteristic is being tallied (see section 7 below), or which data subjects were included in the dataset.
- If the **intersection server** is compromised this only receives individual answers without any context as to who they belong to, or what question they are the answer to. Furthermore every time an answer is queried a random offset is applied by the data server such that the value of the answers will be effectively random and impossible to interpret.

4. Data Processing Workflow

4.1. Survey Completion and Data Encryption

When a user completes a survey each answer is encrypted in the browser using a Javascript implementation of the Paillier Encryption algorithm. Paillier encryption is used in two distinct ways to enable both secure counting and secure logic evaluation:

4.1.1. Encrypted Expanded Bitfield

- **For counting:** Each answer is converted to an expanded one-hot bitfield and encrypted with the reporting server's public key. The homomorphic properties of Paillier encryption allow these encrypted bitfields to be summed directly without decryption, enabling aggregate tallies.
- The bitfield is expanded bitfield with padding bits to prevent overflow during aggregation.
- Example: For a question with five options where the second is selected, the bitfield is `00010`, expanded with padding to `00000000 00000000 00000000 00000001 00000000`.
- The expanded bitfield is encrypted using the reporting server's public Paillier key.

4.1.2. Encrypted Answer Index

- **For logic evaluation:** Each answer is also stored as an encrypted index number using the intersection server's public key.
- Each answer is mapped to a bin number (rather than using a simple zero-based index). This mapping is randomized to obscure which diversity characteristic is being measured, as detailed in Section 7. The mapped bin number is encrypted with the intersection server's public Paillier key.
- The intersection server uses these encrypted indices to evaluate filtering logic (e.g., "is this person's answer in the set {2, 5, 7}?") without learning what the answer actually means.
-

4.2. Data Storage

The encrypted answers are sent by the browser to the data server which stores:

- User identifier (e.g. a secure random GUID).
- Encrypted expanded bitfield for each question: These will be used for counting data subjects
- Encrypted, answer indices: These will be used by the intersection server for evaluating logic expressions to determine individual inclusion/exclusion.

4.3. Why Two Encrypted Formats?

The dual encryption approach might initially seem redundant, but each format serves a distinct and essential purpose:

- The **encrypted bitfield** enables direct aggregation through Paillier's additive homomorphism. When you add encrypted bitfields, you get an encrypted sum of tallies—perfect for counting how many people selected each option, but not suitable for evaluating logical conditions about individual responses.
- The **encrypted answer index** enables the intersection server to perform equality checks and comparisons (e.g., "does this person's answer match option 3?") which are needed for filtering and intersectional queries, but cannot be used for direct counting.

Both formats use Paillier encryption with different server keys, ensuring that neither the reporting server nor the intersection server can independently access both the countable totals and the logical filtering capability for the same individual.

5. Reporting Mechanisms

5.1. Distribution Reporting for a Given Answer

The simplest reporting scenario is reporting on the distribution of answers for a specific question for a specified subset of the population. This consists of

1. Data Retrieval:

- The data server retrieves encrypted expanded bitfields for the specified question and users. The subset of users required is determined by querying an external data store (e.g. HR systems) based on non-sensitive user characteristics e.g. all the staff in department X

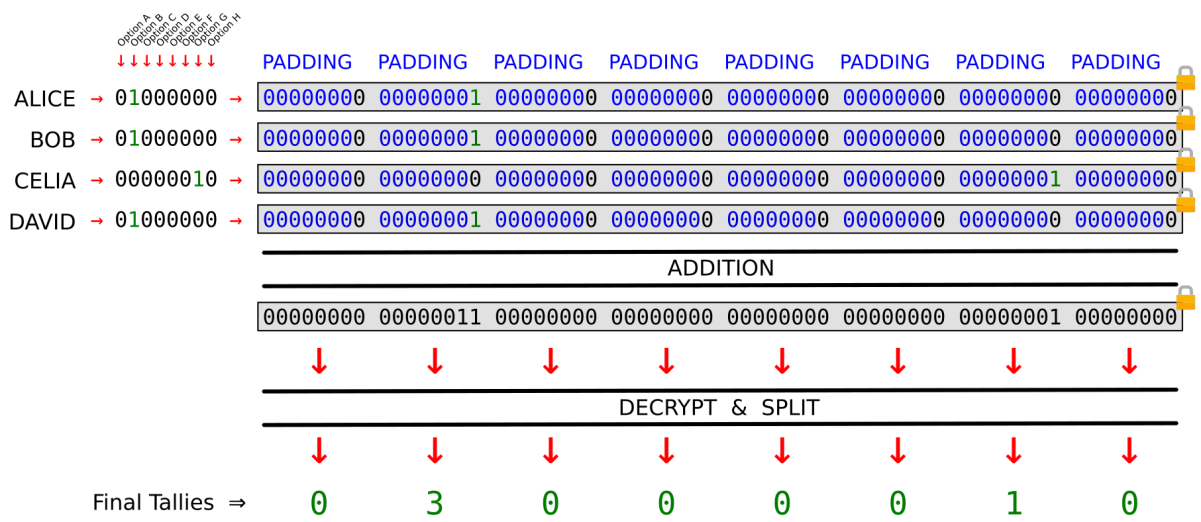
2. Chunking and Aggregation:

- To prevent arithmetic overflow within the Paillier encryption scheme, responses are divided into manageable bundles (chunks) of data subjects before summation. For example, with 1,000 responses, we might process them as two chunks of 500 subjects each. The maximum chunk size depends on the padding space allocated in the bitfield.
-
- Paillier addition is used on the data server to sum encrypted expanded bitfield answers for all users within each chunk. Crucially, this summation happens entirely on encrypted data—no decryption occurs at this stage.
- The manner of storage means that simply summing all the bitfields for all users will result in a set of separate tallies for each answer option. The padding provides the space for these tallies to count up without interfering with each other.

3. Decryption and Tallying:

- The reporting server decrypts the chunk sums. In the basic implementation, these could be returned as plaintext to the data server. However, as described in Section 6 and Section 7, a more complete implementation can use requester Paillier key pairs to encrypt the decrypted tallies before returning them to the data server. This ensures the data server never sees plaintext counts and can only work with encrypted values.
- The data server sums the results across all chunks to compute the final tally for each answer option. When using requester Paillier keys (Section 7), this final summation is performed on encrypted tallies using Paillier's homomorphic properties.

The process of padding, and summing the user answers to obtain the final tallies is illustrated below.



5.2. Key Length And Bundle Size

When encoding survey responses into Paillier ciphertexts, we pack multiple answer options for a single question into one plaintext value. Each ciphertext is specific to one question; the full plaintext space of the modulus is therefore available per question.

We define the following parameters:

- **n**: the Paillier modulus (for a 2048-bit key, $A \cdot g < \lfloor \log_2 n \rfloor$).
- **A**: the maximum number of answer options for the question.
- **M**: the maximum number of individual responses combined into a single bundle.
- **g**: the number of bits allocated to each answer slot.
- **B = 2^g**: the radix used to separate slots in the packed plaintext.

Each slot i (for answer option i) then encodes an integer count $c_i \leq M$. The packed plaintext is of the form

$$\sum_{i=0}^{A-1} c_i B^i.$$

To ensure correct operation, the following conditions must hold:

1. **Slot capacity:**

$$g \geq \lceil \log_2 M \rceil + s,$$

where s is a small safety margin (2–4 bits) to accommodate bundle size variation, coding errors, or modest future increases.

2. **No slot overlap:** adjacent slots are separated by factors of B^i , so provided the above holds, values in one slot cannot carry into the next.

3. **No modulus wrap:**

$$A \cdot g < \lfloor \log_2 n \rfloor.$$

5.2.1. Worked example

Suppose we allow up to **A = 40** answer options per question, and combine up to **M = 10,000** responses per bundle. Then $\lceil \log_2 M \rceil = 14$. With a slack margin $s = 2$, we set $g = 16$. The total bit requirement is

$$A \cdot g = 40 \times 16 = 640 \text{ bits.}$$

Since the modulus provides ~2048 bits, this uses less than one-third of the available space. The remaining ~1400 bits give ample margin for future increases in bundle size or answer options.

5.2.2. Summary

To put it another way: even with quite generous assumptions - as many as forty answer options per question and ten thousand responses per bundle - the encoding scheme consumes only a small fraction of the capacity available in a 2048-bit modulus. There is more than enough room for safe packing, operational variation, and future extension. In

practice, the cryptographic packing limit is not a binding constraint; bundle size can be chosen flexibly to balance efficiency and privacy.

5.3. Intersectional Reporting

There is often a requirement for more complex forms of reporting where the population being counted depends not just on some non-sensitive attribute, but also on one or more of the encrypted characteristics e.g., "ethnicity breakdown for all men in department X". This requires a more involved process as follows::

1. Logic Expression Formation:

- The requester provides a logic expression using question IDs and desired answers e.g. "(question_3_answer == option_1 AND question_7_answer == option_4) OR (question_9_answer == option_2)"

2. Data Preparation:

- The data server retrieves the encrypted answers indices relevant to the logic expression.
- The data server generates a unique random offset for each user for each question in the logic expression, and applies these offsets using Paillier addition. For example, if the expression includes "question_3_answer == option_1" and a random offset of 2734 is chosen for this specific user's answer to question 3, then Paillier addition adds 2734 to the encrypted answer, and the logic expression is adjusted to "question_3_answer == option_2735". This ensures that the intersection server never sees the actual answer values. While the intersection server can identify when multiple users have the same answer to a question (because the offset-adjusted values will match), it cannot determine what that answer actually is or correlate it with other questions.
- This adjustment also serves to re-randomise the answer ciphertext.
- The data server also sends a re-randomised (i.e. add zero to the answer using Paillier addition) version of the expanded bitfield for the question being reported on. The re-randomisation prevents the intersection server from recognising when the same ciphertext is re-submitted for different expressions.

3. Logic Evaluation:

- The intersection server decrypts the offset-adjusted answer values and evaluates the logic expression.
- If the expression is true, the intersection server returns a re-randomised version of the encrypted expanded bitfield that was sent for this data subject. Re-randomization is achieved by using Paillier addition to add zero to the existing ciphertext, which changes its encrypted form without changing its value. The intersection server does not generate a new ciphertext from scratch; it only re-randomizes the bitfield that was already provided.
- If the expression is false, a freshly generated encrypted zero (encrypted with the reporting server's key) is returned instead.
- This process ensures that the data server cannot infer from the response whether the criteria was met or not for that data subject, as both true and false results appear as re-randomized ciphertexts.

4. Aggregation and Reporting:

- The data server aggregates responses as in distribution reporting discussed above.
- Note that in this case, any data subject who falls outside the specified intersection will now have a zero bitfield so will not count towards the tally in any way. Thus the final tally reflects only those individuals meeting the specified criteria.

5.4. Handling Multiple Choice Questions

For questions allowing multiple selections:

- Each possible answer is treated as a separate binary (yes/no) question for the purposes of intersectional reporting
- When it comes to the expanded bitfield, all selected options can be set to 1.
- Aggregation and reporting follow the same procedures.

6. Additional Safeguard: Requester Key Pairs And Encrypted Results

An additional security measure can be introduced by equipping each legitimate reporting requester with their own asymmetric RSA key pair:

6.1. Request Signing

Each reporting request is signed by the requester using their private key.

The requester also provides their public key signed by a trusted central certifying authority. This allows both the data server and reporting server to check that the request has not been tampered with and verify that the request originated from an authorised entity.

6.2. Encryption Of Tallies For The Requester

The requester's certified public key can also be used to encrypt the tallies produced by the reporting server to be sent back to the requester. This ensures that only the requester can decrypt the final results.

If the data server is compromised, an attacker might attempt to hijack legitimate queries and substitute their own sets of individuals in order to compare outputs from different overlapping subsets (a difference attack). Encrypting the results with the requester's key prevents the attacker from seeing the decrypted tallies. While such hijacking could still disrupt the reporting process, it would not reveal any personal data to the attacker.

This approach requires a shift in processing responsibility: the summing and tallying of decrypted results is performed by the requester rather than the data server. However, in the next section we discuss an approach which involves assigning each requester a Paillier key pair (for encryption of reporting data) in addition to an RSA keypair (for signing reporting requests). This would allow summing of the tallies on the data server.

7. Protecting Against Inference In Aggregate Reporting

7.1. Inferring The Statistic Being Reported On

The approach set out above leaves a potential inference attack on the reporting server insofar as an attacker with access to that server could still infer what statistic is being measured. This is because the number of possible answers to a question, or the shape of the frequency distribution, often reveals which diversity characteristic is being tallied e.g. if there are counts for just three answers then this might suggest gender.

7.2. Remapping Answer Bins

To address this, we can build in an additional safeguard to disguise the “shape” of the data. Instead of placing answers directly into their true categories (answer 1 => 1, answer 2 => 2 etc) each answer is mapped into one of a fixed set of randomised bins (answer 1 => 15, answer 2 => 5 etc). The mapping is random insofar as answer 1 might be mapped to bin, say, 15, but constant in that these mappings are set once at system creation and then do not change for a given question over time. An answer for a given question from any user at any time will always use the same mapping. The mappings are arranged such that for all questions the total number of bins is the same - say 20. This means that for gender, an answer of ‘male’ could be represented by a count of 1 in any of 7 different bins, ‘female’ would map to another 7 bins and ‘non-binary’ to another 6 bins. If an answer maps to more than one bin, then just one of these is picked at random when the user’s answer is stored. This means that from the reporting server’s perspective, every question looks the same - with, say, 20 bins and no obvious link between bins and real answers. The underlying counts can still be reconstructed later by recombining the bins according to the hidden mappings.

7.3. Long Term Mapping Inference

There remains a residual risk: a determined attacker could attempt long-term statistical analysis, comparing distributions across time to make educated guesses about which bins correspond to which real answers. To reduce this risk, we extend the approach by defining multiple bin mappings. Each user’s response is assigned to one of these mappings at random on creation in the browser. The data server records the Paillier-encrypted bin-mapped answer along with the ID for the mapping that was used when it was stored. The data server groups results by mapping and sends these to the reporting server for decryption. When the data server gets the tallies back, the bins can be recombined into the true answer categories. This makes it much harder for an attacker to link bin distributions back to underlying diversity questions.

7.4. Problem With RSA Encrypting Reporting Data

A further complication arises because, in order to regroup bins into the final answer tallies the reporting server must be able to take the tallies for any given group and add together all the bins that represent a single answer (e.g. all the bins that could be used for “male”). If the reporting data is encrypted using the requester’s public RSA key then the data server would not be able to see the numbers it needs to add together. The solution is to use homomorphic

encryption to encrypt the frequency distributions sent back to the requester. Each requester has a Paillier keypair in addition to an RSA keypair. The reporting server encrypts the tallies it returns with the requesters public Paillier key. This allows the data server to recombine the counts while still encrypted. With this in place, the data server can regroup bins into their original categories without ever seeing the underlying numbers. The final tallies are then decrypted only by the authorised recipient (the original requester) using their private Paillier key.

7.5. Benefits of this Approach

This layered approach - random binning, multiple mappings, and homomorphic regrouping - ensures that the reporting server never learns the structure of the data it holds, while recipients still receive accurate answer-level totals. It significantly raises the bar against inference attacks, providing strong protection for sensitive diversity information.

8. Defence Against Differencing Attacks

While the system described above secures data against the compromise of any single server, it does not inherently protect against *re-identification via differencing attacks*. Even if safeguards were introduced to block reporting on small groups of individuals, an attacker who could make arbitrary queries (as legitimate requesters are intended to be able to do) could still infer individual characteristics.

For example, an attacker could issue a query over a cohort of 100 individuals (Query A), then re-run the same query including one additional individual (Query B). By subtracting the results of Query A from Query B, the attacker could isolate the characteristics of the extra individual. Thus, compromising a single machine with the ability to submit arbitrary queries would allow systematic re-identification, nullifying other protective measures.

This risk is partly reduced by the fact that user cohorts might ultimately (in a real world implementation) be defined by characteristic-based filters rather than explicit lists of individuals. However, with carefully crafted overlapping queries, an attacker could still approximate difference-based re-identification.

Two main approaches exist for defending against such attacks:

a. Query Overlap Detection and Blocking

- Track all historical queries and block new ones that overlap in ways that might enable re-identification.
- This requires sophisticated logic to detect whether arbitrary combinations of past and present queries could isolate an individual.
- This approach is complex, computationally expensive, and may cause confusing rejections of seemingly innocuous queries, damaging usability.

b. Adding Noise via Differential Privacy (DP)

- Introduce carefully calibrated noise to reported results so that individual-level information cannot be reliably inferred.
- This approach is well-established, simpler to implement (or at least there are existing off-the-shelf implementations), and allows users to issue arbitrary queries.
- Queries relating to very small groups will naturally produce results dominated by noise, effectively preventing useful re-identification. With very small data sets the noise will swamp the underlying data making the result effectively useless - but this is fundamental to the operation of DP.

8.1. Where and How to Add Noise

Noise could be introduced at different stages of processing:

- **On the reporting server:** noise is added directly to decrypted tallies before returning results. Since the server knows the cohort size, it can determine appropriate noise levels and apply them to each tally.
- **On the data server:** noise is added by injecting synthetic encrypted responses into the aggregation process.

- **Split between servers:** to protect against compromise of either the data or reporting server, both could independently contribute a share (e.g. 50%) of the noise. This ensures that if one server is compromised and the attacker strips out its noise contribution, the final results still retain some uncertainty.

8.2. Deterministic Noise Strategy

A further refinement is to generate noise deterministically based on a hash of the user IDs in the query combined with the date. Combined with a batching process that incorporates new responses only once per day (e.g. at midnight), this would have two advantages:

1. **Consistency within a day** – the same query always produces the same noisy result, avoiding user confusion.
2. **Natural variation over time** – the same query on subsequent days produces different noisy results, preventing attackers from isolating newly added responses and providing an additional privacy layer.

8.3. Work In Progress

Our current intention is to implement DP on the data server, but the exact implementation has not been determined at this time. The practicalities of where and how to implement differential privacy within this framework will be investigated in the next phase of the project and this paper updated later in the project to reflect the findings.

9. Security Analysis

9.1. Threat Model and Assumptions

- **Semi-Honest Adversary Model:** The system assumes that all servers follow the protocol as specified but may attempt to learn additional information from the data they observe. The system does not provide protection against active adversaries who arbitrarily deviate from the protocol (for example, a data server that manipulates ciphertext values or fabricates responses). Achieving active security would require additional mechanisms such as zero-knowledge proofs to verify correct computation, which are noted as potential future enhancements in Section 13.
- **Server Compromise:** The system assumes that no more than one server can be compromised at a time.
- **Non-Collusion:** Servers are operated by separate agencies with strict controls to prevent collusion.
- **Noise and Utility Trade-Off:** With the adoption of Differential Privacy (DP), re-identification via small cohort queries is not possible. However, results for very small groups will inevitably be dominated by noise, making them unreliable. This is an intentional trade-off to ensure privacy protection.

9.2. Challenges of Formal Security Proofs

Whilst the mathematical complexity of the cryptographic primitives used in this system (primarily Paillier encryption) is lower than some more sophisticated schemes such as fully homomorphic encryption or advanced multi-party computation protocols, the architectural design described here presents significant challenges for formal security verification especially if there was a desire to formally prove security in the case of a malicious adversary as opposed to our current semi-honest threat model.

The multi-server architecture, and in particular the role of the intersection server in evaluating arbitrary logical expressions over encrypted data, creates a non-standard configuration that does not readily map to existing security proof frameworks. Most formal cryptographic security proofs rely on simulation-based arguments within well-defined threat models (typically semi-honest or malicious adversaries in two-party or simple multi-party settings). Our system's three-server architecture with its specialized division of responsibilities—where the data server orchestrates queries, the intersection server evaluates filtering logic, and the reporting server performs decryption—does not fit neatly into these standard frameworks.

This complexity is driven by the fundamental problem the system addresses: enabling arbitrary intersectional reporting on encrypted diversity data. The ability to compute statistics over flexible, user-defined intersections—such as counting individuals matching "(religion X OR religion Y) AND (sexuality Z OR disability P)"—after the data has already been encrypted and stored, is an inherently challenging problem in privacy-preserving computation.

If the required intersections were known in advance at the time of data collection, standard cryptographic approaches could be applied. However, the requirement to support arbitrary,

ad-hoc intersectional queries specified long after data collection introduces significant complexity. To our knowledge, no "standard" cryptographic protocol currently exists that can efficiently handle this use case while maintaining the level of query flexibility and comprehensibility required for practical diversity reporting.

Developing a formal security proof for this system would therefore require either:

- Extending existing proof frameworks to accommodate the three-server architecture and arbitrary query model, or
- Simplifying the system to fit existing frameworks, which would likely require sacrificing either query flexibility or the separation of concerns that provides single-server compromise resistance.

For the purposes of this work, we have prioritized practical security properties (encryption of individual responses, separation of decryption capabilities, differential privacy protections) over formal provability. We acknowledge that this approach relies more heavily on the semi-honest adversary assumption and organizational controls than a system with a complete formal security proof would require. This trade-off reflects our judgment that, for the diversity reporting use case, practical deployability and comprehensibility are at least as important as formal mathematical guarantees.

9.3. Partially Mitigated Attacks

The following scenarios outline attacks that this approach cannot fully protect against:

9.3.1. Collusion Between Servers

Risk: If two or more servers collude, they could potentially decrypt individual responses.

Mitigation: Organisational controls, separate hosting, and legal agreements between the agencies responsible for the separate systems reduce but cannot eliminate this risk.

9.3.2. Multiple Server Compromise

Risk: The system cannot withstand simultaneous, or even sequential (if unnoticed), compromise of more than one server.

Mitigation: Organisational controls, separate hosting, intrusion detection, and strict breach reporting requirements are relied upon here.

9.3.3. Active Adversaries

Risk: A server that actively deviates from the protocol (for example, a data server that manipulates ciphertext values to skew results, or fabricates encrypted responses) could compromise the integrity of reported statistics without necessarily revealing individual responses.

Current Status: The system is not designed to detect or prevent active adversaries. While confidentiality of individual responses is maintained through encryption, a malicious server could potentially manipulate aggregated results.

Mitigation: Achieving active security would require additional cryptographic mechanisms such as zero-knowledge proofs to verify that servers perform computations correctly, or verifiable computation schemes. These would significantly increase system complexity and computational cost, and are noted as areas for future research in Section 13.

10. Mitigated Attacks Under Semi-Honest Assumption

Under the semi-honest adversary model (where servers follow the protocol but may attempt to learn information from their observations), the following threats are mitigated by the technical measures described in this paper:

10.1. Data Server Compromised to Access Stored Data

Risk: Attacker reads the database.

Mitigation: Responses are stored under Paillier encryption; private keys are held off the data server. No plaintext individual responses are present. New responses are encrypted on the client before they reach the server

10.2. Reporting Server Compromised to Access Decryption Key

Risk: Attacker obtains the reporting server's private key.

Mitigation: The reporting server only handles aggregated sums (never per-subject values) and lacks context about who/what is being tallied.

10.3. Data Server Compromised to Submit/Hijack Arbitrary Requests

Risk: Attacker crafts or hijacks queries to run chosen cohort analyses.

Mitigations:

- **Signed requests:** Every reporting request must be signed by a legitimate requester; the data server cannot invent valid requests.
- **Requester-only visibility:** Results are encrypted to the requester's public key; without the requester's private key the attacker cannot read outputs.
- **DP noise:** Even if query flow is hijacked, small-cohort results are noise-dominated, defeating differencing attacks.

10.4. Requesting System Compromised (Arbitrary Queries)

Risk: A legitimate requester's system is taken over, enabling arbitrary queries.

Mitigation: DP prevents differencing/attribute inference across overlapping cohorts; small groups yield unusable, noise-dominated outputs. Optional rate-limits, query budgets and auditing further constrain abuse.

10.5. Requesting System Compromised (Many Queries to Average Out Noise)

Risk: Attacker issues many repeats to "average away" noise.

Mitigations:

- **Deterministic per-day noise** (seeded by cohort hash + date) yields one noise instance per day.
- **Midnight batching** of new responses prevents learn-as-you-go membership inference; composition controls (query budgets) limit cumulative privacy loss. Within a day, repeats don't give independent samples; across days, noise changes, blocking convergence on true values.

10.6. Intersection Server Compromised

Risk: Attacker sees decrypted values for logic checks or links responses across queries.

Mitigation: The data server applies random offsets to answer indices and re-randomises ciphertexts before each evaluation; the intersection server returns only a re-randomised encrypted bitfield (or zero) and has no subject/question context. This prevents learning true answers or linking a subject across requests.

10.7. Replay/Linkability of Ciphertexts on Intersection Server

Risk: Linking the same subject's ciphertext across multiple queries to build profiles.

Mitigation: Systematic re-randomization of ciphertexts (and fresh offsets for intersection checks) breaks linkability across requests.

10.8. Presence/Participation Inference (Detecting New Responses)

Risk: Observers detect that "someone new" has submitted a response by watching day-to-day deltas.

Mitigation: Daily deterministic noise + midnight ingestion of new responses means results change daily regardless of new participation; observers cannot tell whether new responses were added, let alone who.

10.9. Decryption-Oracle Abuse

Risk: Treating servers as decryption oracles by crafting special inputs.

Mitigation: The reporting server only decrypts aggregated sums via verified, signed requests and returns results encrypted to the requester; the intersection server never returns decrypted plaintexts – only (re-)encrypted selectors – so neither can be used as a general decryption oracle.

10.10. Request Tampering / Injection in Transit

Risk: Altering a query en route to change cohorts or reporting dimensions.

Mitigation: End-to-end request signing/verification ensures authenticity and integrity of requests accepted by the data/reporting servers.

11. Performance And Scalability

Paillier encryption is simpler and less computational intensive than other fully homomorphic encryption systems. However, it is still more intensive than a conventional database lookup.

We can mitigate this to some extent by using Intel's Paillier implementation with support for the AVX512 IFMA instruction set (<https://www.intel.com/content/www/us/en/developer/articles/technical/homomorphic-encryption/iso-compliant-paillier-cryptosystem-library.html>). As well as providing a robust and security-audited implementation of the Paillier cryptosystem, this also provides hardware acceleration for some of the more intensive maths operations. The AVX512 extensions are available in most Cloud hosting environments.

12. Comparison with Alternative Approaches

To contextualise our proposed architecture, it is useful to compare it against other approaches that might be considered for storing and processing sensitive diversity data. The table below evaluates various privacy-preserving systems across five critical dimensions relevant to real-world deployment.

- **Single-server resistance** measures whether the system can withstand compromise of any individual server whilst still protecting individual records - a crucial property given that server breaches represent the primary threat in practice.
- **Query flexibility** captures the ability to perform arbitrary analyses after data collection, including intersectional queries that were not anticipated at design time.
- **Performance** reflects the computational efficiency of query execution.
- **Comprehensibility** indicates whether the system's security model can broadly be explained to non-technical stakeholders - a key factor in building trust and adoption.
- **Running cost** encompasses both computational resources and operational complexity.

Approach	Single-Server Resistance	Query Flexibility	Performance	Comprehensibility	Running Cost
Symmetric Encryption	✗ None	✓ High	✓ Excellent	✓ High	💰 Low
Single-Server HE	✗ None	✓ High	✗ Poor	⚠ Medium	💰💰 Medium
TEE-based	✗ None*	✓ High	✓ Good	✗ Low	💰💰 Medium
This System	✓ Strong	✓ High	⚠ Moderate	⚠ Medium	💰💰💰 Medium
Full MPC	✓ Strong	✗ Low	✗ Poor	✗ Low	💰💰💰💰 Very High
Threshold HE	✓ Strong	✓ High	⚠ Moderate	✗ Low	💰💰💰 High

It is worth noting that trusted execution environments (TEEs), whilst protecting cryptographic keys from extraction, do not prevent a compromised host from misusing the legitimate decryption interface to extract individual records - a limitation that makes them equivalent to standard server-side encryption for this threat model. Similarly, whilst full multi-party computation offers strong theoretical guarantees, its computational and communication overheads make it impractical for the interactive, exploratory analyses required in diversity reporting.

As the comparison illustrates, our three-server Paillier-based architecture occupies a pragmatic middle ground: it provides genuine resistance to single-server compromise whilst maintaining the query flexibility essential for diversity analytics, at a computational cost that we hope we will be able to demonstrate, will be feasible for typical organisational deployments.

13. Next Steps

The next phase of this project will focus on developing an open-source, end-to-end proof of concept that implements the full workflow described in this paper, from in-browser encryption of survey responses through to secure reporting. This prototype will make use of Intel's Paillier library to ensure that the cryptographic foundations are robust, security-audited, and capable of taking advantage of the AVX512 hardware acceleration already discussed.

By building a working system, we will be able to explore how the proposed architecture behaves in practice. The proof of concept will cover the entire data lifecycle: survey completion and client-side encryption, secure storage on the data server, request handling with requester signatures, aggregation with added differential privacy noise, and final decryption of tallies by authorised requesters. Running this on cloud-hosted servers with AVX512 support will allow us to measure both the speed of operations such as encryption, decryption, and aggregation, and the time taken to generate complete reports for realistic population sizes.

Equally important will be the ability to measure the operational cost of running the system at scale. Cloud-based benchmarking will provide concrete figures for both performance and cost, enabling us to determine whether the architecture is viable in practice for production use. The outcomes of this stage will inform decisions about optimisation, potential architectural refinements, and whether the system can realistically support the reporting requirements of organisations that need both privacy protection and flexible access to diversity data.

13.1. Deferred Enhancements and Limitations

The approach outlined in this paper is intended as an incremental improvement over current practice, not a final or complete solution. Budget constraints, ease of implementation, and deployability have all shaped the scope of this work. We have therefore noted below, a number of areas where stronger cryptographic techniques are possible, but have been not been included in this project:

- **Threshold Paillier**

Splitting the decryption key across multiple servers would remove single-point key compromise. This is a natural next step, but requires more complex key management and additional infrastructure. The current design can be enhanced later to incorporate threshold techniques without invalidating the basic architecture.

- **Zero-knowledge proofs of input correctness**

At present, a malicious client could submit malformed responses (e.g. multiple options ticked for a single-choice question). Zero-knowledge proofs would allow servers to verify one-hot or range constraints without learning the underlying answer. More broadly, zero-knowledge proofs represent a pathway toward active security: they could verify that servers perform computations correctly without revealing sensitive data, addressing some of the active adversary limitations noted in Section 9.2.3. Zero-knowledge proofs for validating encrypted inputs (such as verifying that a Paillier-encrypted value represents either 0 or 1) are relatively efficient compared to other cryptographic techniques. However, implementing such proofs is currently out

of scope due to the added complexity and reduced comprehensibility they would introduce.

As an interim mitigation, the reporting server could periodically validate a random subset of submitted answers. Specifically, the data server could forward a small fraction of encrypted responses to the reporting server for direct decryption and correctness checking. These would be presented completely out of context so that it was not possible to infer the meaning of, or any relationship between, the answers being decrypted. Over time this would create statistical assurance that invalid submissions are rare, while ensuring the reporting server only sees a scattered and uncorrelated stream of individual answers. Although this approach cannot guarantee correctness in every case, it could provide a pragmatic balance until ZK proofs can be implemented.

- **Differential Privacy configuration**

We have not attempted to reproduce the extensive literature on differential privacy in this paper. Any practical implementation must nevertheless take care to:

- choose an appropriate noise mechanism (e.g. discrete Laplace or Geometric),
- set and manage privacy parameters ϵ and δ ,
- use a privacy accountant to track cumulative exposure,
- employ a secret-keyed PRF if day-stable noise is required,
- canonicalise queries to avoid gaming, and
- enforce per-entity privacy budgets.

- **Client-side hardening**

As noted earlier in the *Scope of the Paper* section, all web-based systems remain exposed to the risk of compromised script delivery. Established mitigations (subresource integrity, content security policy, signed bundles, etc.) are well documented, but are not covered here. The limitation is inherent to browser-based deployment regardless of the approach used to store the data that is being collected.

14. Conclusion

We think that the system described in this paper offers a significant advance in the way sensitive diversity survey data can be collected and reported. At its heart, it delivers strong security by ensuring that individual responses are never exposed in plaintext and remain protected even during aggregation and also in the event of a single-server compromise. By combining encryption with organisational controls, it becomes possible to safeguard participant data without undermining the ability to analyse it.

At the same time, the design has been shaped to preserve the flexibility that organisations need. Detailed and intersectional reporting remains possible, allowing reporting users to gain meaningful insights into their workforce while still respecting individual privacy. This balance between privacy protection and analytical power is central to making the system practically usable.

The architecture is also grounded in practicality. Rather than relying on highly complex or computationally prohibitive fully homomorphic encryption, it uses Paillier encryption together with carefully chosen safeguards such as request signing and differential privacy. This keeps implementation complexity manageable while still addressing the most important risks. By

focusing on solutions that are technically feasible, the system stands a realistic chance of being adopted and deployed in real-world environments.

Finally, the approach demonstrates a visible commitment to privacy. Investing in privacy-preserving technology sends a strong signal to survey participants and to wider stakeholders that data is being handled responsibly. This, in turn, can help to build trust, foster greater engagement with surveys, and ultimately improve the quality and completeness of the diversity data collected.